

SESSION 1

Introduction to Context Engineering

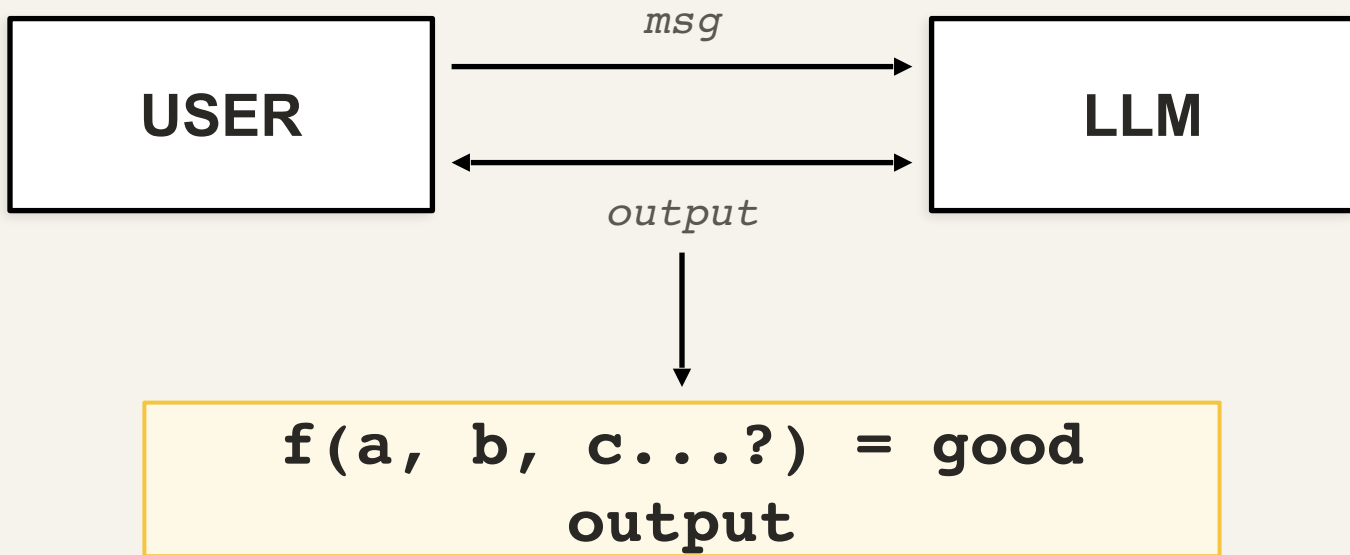
From prompts to the systems that control
what information reaches the model.

Hands-On Context Engineering

O'Reilly Live Training · Lucas Soares

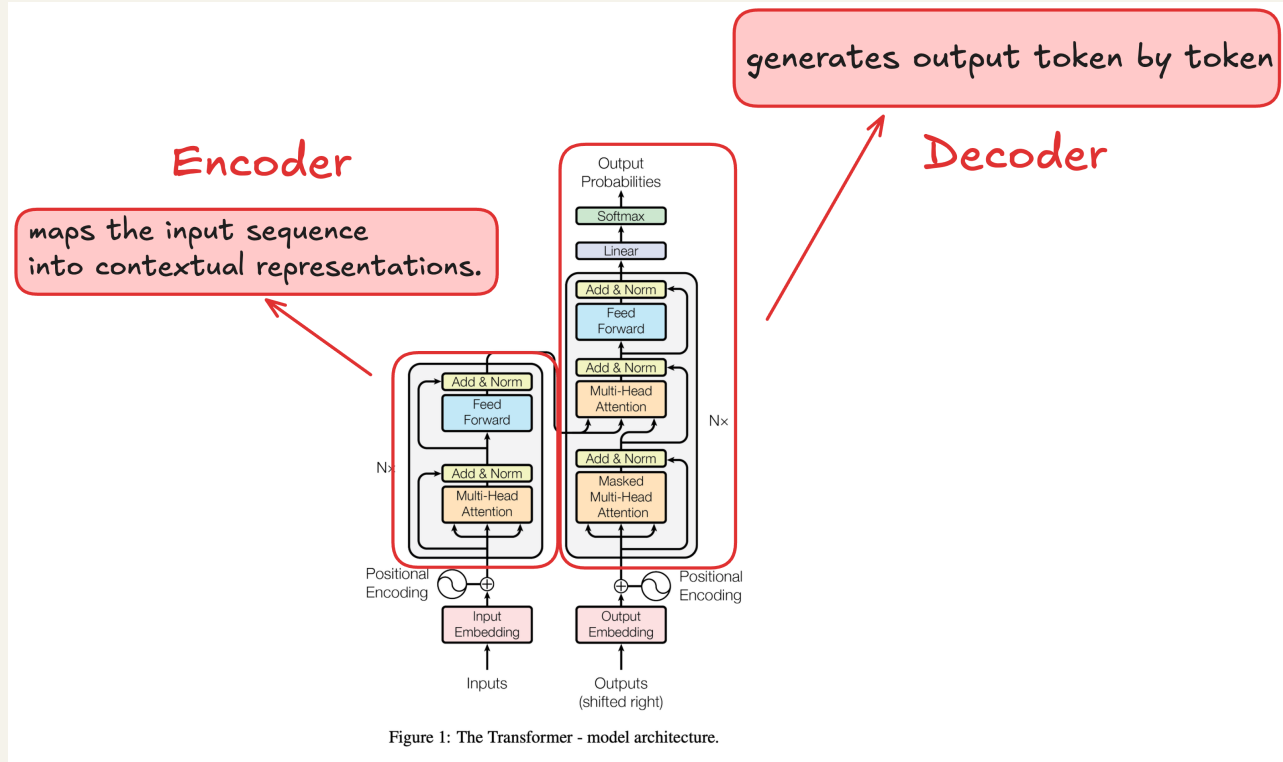
How to get good outputs from LLMs?

How to get good outputs from LLMs?



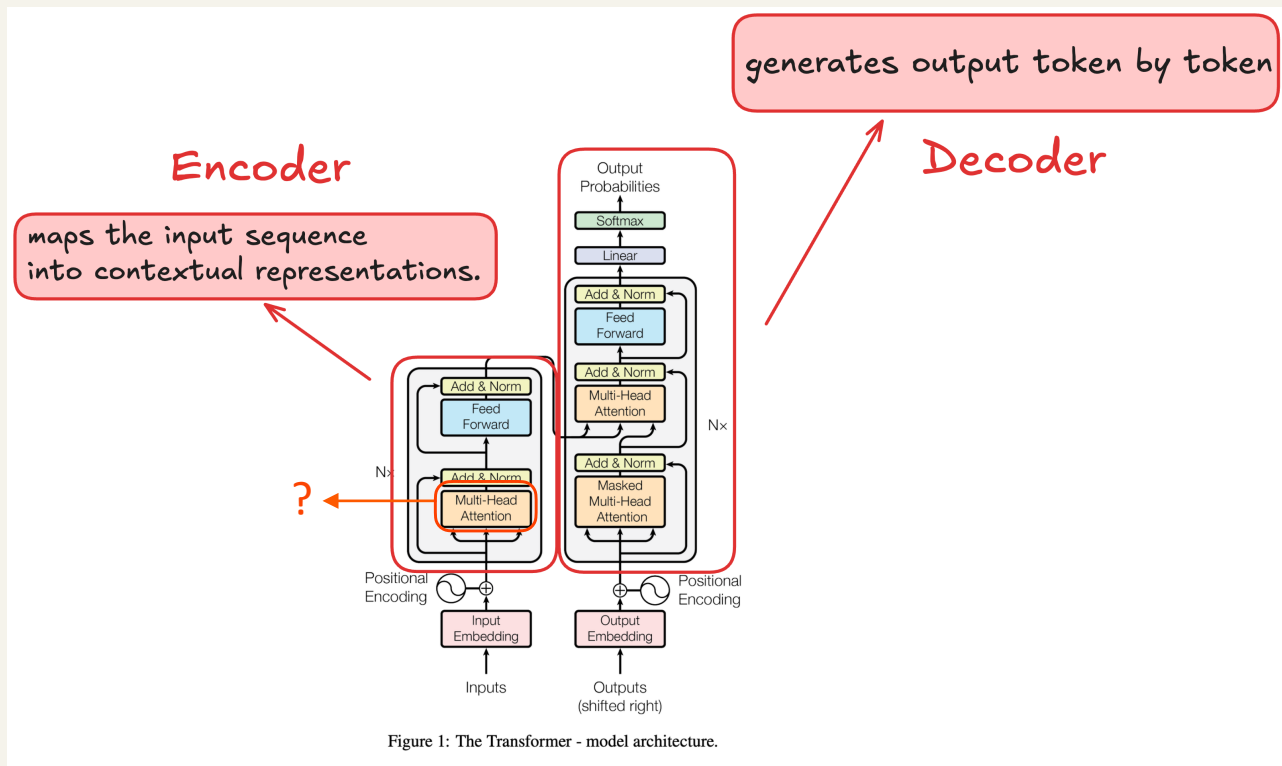
Good outputs are a function of what?

Encoder/Decoder Architecture



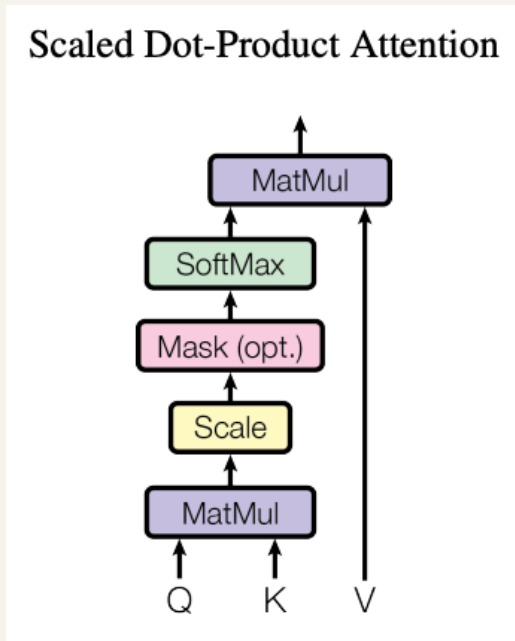
<https://arxiv.org/pdf/1706.03762>

Encoder/Decoder Architecture



Attention Mechanism

Scaled Dot-Product Attention



"Given token X, which other tokens in the sequence are most relevant to understanding it?"

Query

what the token is looking for

Key

what each other token advertises about itself

Value

the information each token can contribute

Attention Mechanism

Who is relevant to whom?

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

How much attention budget goes to each?

What do I actually take from those I attend to?

So the model does not treat every token equally.
It assigns more weight to tokens that match the current query more strongly.

Attention Mechanism

Attention Weights are a Map of Relevance

		Key		
		the	cat	sat
Query	the	A_{11}	A_{12}	A_{13}
	cat	A_{21}	A_{22}	A_{23}
	sat	A_{31}	A_{32}	A_{33}

relevance of token i to token j

Long inputs create many competing possible relationships

The paper gives self-attention a short path length advantage, but Table 1 also shows per-layer complexity of $O(n^2 \cdot d)$.

n tokens $\rightarrow O(n^2 \cdot d)$

- **Benefit:** any token can connect to any other token with a short path length.
- **Tradeoff:** as sequence length grows, the model must score many more possible pairwise relationships.
- **Classroom point:** "more context" adds candidates for attention — not just more useful evidence.

Table 1 comparison from the paper

Self-attention has path length $O(1)$, but per-layer complexity grows with sequence length.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size and r the neighborhood size in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Max Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$

A long prompt can still fit inside the context window and yet be hard to use well if the relevant parts are diffuse, buried, or competing with noise.

$f(a,b,c\dots)$ for good outputs

LLM $\xrightarrow{?}$



token by token
generation

Order matters!

n tokens $\rightarrow O(n^2 \cdot d)$

more context
equals more
candidates for
attention



Process maps
of relationships

A long prompt can still fit
inside the context window
and yet be hard to use well if
the relevant parts are
diffuse, buried, or competing
with noise.

The Wall Every Developer Hits

You start with a model that can write, summarize, and reason brilliantly. Then you apply it to a real problem.

NO PRIVATE KNOWLEDGE

Can't answer questions about your documents, codebase, or users.

NO MEMORY

No knowledge of yesterday's events or your last conversation.

CONFIDENT HALLUCINATIONS

Confidently fabricates answers when it doesn't know.

What is Context Engineering?

The discipline of **designing the architecture** that feeds an LLM the **right information** at the **right time**.

THE CORE PRINCIPLE: ENTROPY REDUCTION

Context engineering transforms **high-entropy** human intentions into **low-entropy** representations machines can process. Unlike humans, machines cannot "fill in the gaps."

NOT JUST PROMPTS

The entire information system around the model

NOT JUST RAG

Retrieval, memory, tools, and orchestration

NOT STATIC

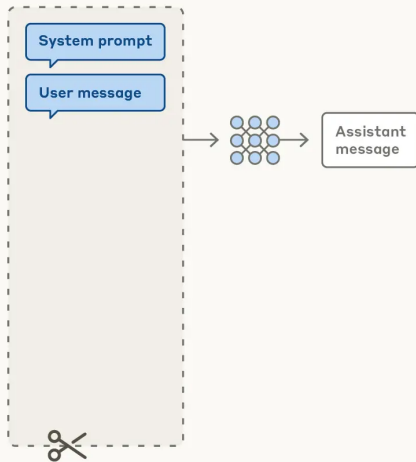
Context must be dynamically managed and pruned

Context Engineering vs Prompt Engineering

Prompt engineering vs. context engineering

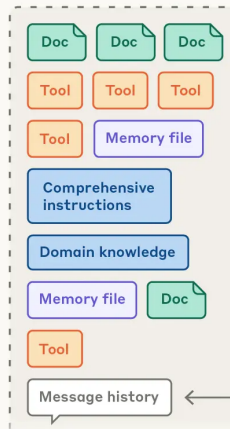
Prompt engineering for single turn queries

Context window

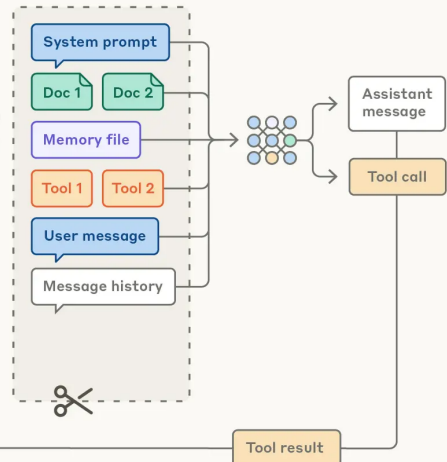


Context engineering for agents

Possible context to give model



Context window

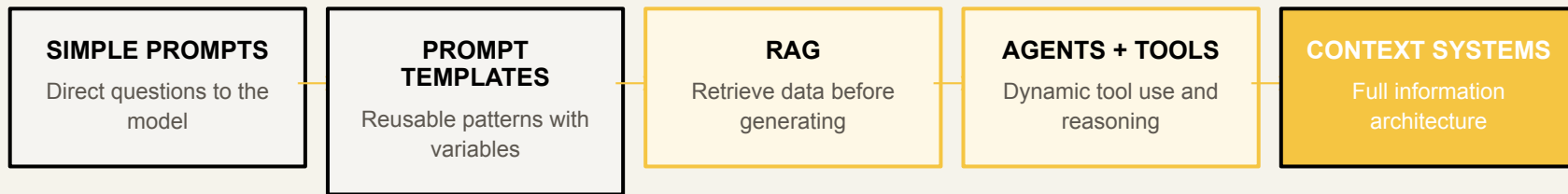


SECTION 02

The Evolution

From simple prompts to full context systems

From Prompts to Context Systems



Each step added more sophistication to **how we manage information flow**. Context engineering encompasses all of these.

Storage, Management, Usage

Everything in context engineering falls into one of three activities.

01

Storage

Collecting and persisting context

- RAG pipelines and retrieval
- Tool outputs and API results
- User feedback loops
- External knowledge bases

02

Management

Organizing and maintaining context

- Chunking and structuring
- Pruning stale information
- Priority-based inclusion
- Memory architecture

03

Usage

Deploying context strategically

- Prompt construction
- In-context learning
- Dynamic tool selection
- Structured outputs

SECTION 03

The Context Management Challenge

Understanding the constraints that shape every AI system

The Context Window

The context window is the model's **active working memory** -- the finite space where it holds instructions and information for the current task.

Every word, number, and punctuation mark consumes tokens. Once full, older information gets erased.

Think of it like a whiteboard: once full, you must erase to write new things. Important details can vanish.

CPU = LLM
The reasoning engine

RAM = Context Window
Active working memory

Disk = External Storage
Vector DBs, files, APIs

Key insight: "Just like a laptop with too many browser tabs open, this RAM can fill up fast."

The Context Budget Problem

Every agent must make critical decisions about its finite context budget.

KEEP ACTIVE

What stays in the context window right now?

STORE EXTERNALLY

What goes to disk or DB for later?

COMPRESS

What can be summarized to save space?

RESERVE

How much capacity for reasoning?

This is the core design challenge. Get it right and your agent is reliable. Get it wrong and everything breaks.

Bigger Windows $\neq$ Better

Models now support 100K, 200K, even 1M+ tokens. But longer contexts actually introduce **new failure modes**.

Performance degrades far before reaching maximum capacity.

"Larger models do not necessarily create the best AI systems -- better engineering does."



Accuracy Drops

Irrelevant info crowds out what matters



More Hallucinations

Models become less grounded



Contradictions

Long contexts contain conflicting info



Higher Costs

Every token incurs compute cost

Four Context Failure Modes

As context grows, these failure modes emerge:

POISONING

Incorrect info enters context and **compounds over time** as the agent reuses it.

DISTRACTION

Too much history causes the agent to **repeat past behavior** instead of reasoning fresh.

CONFUSION

Irrelevant tools or documents **crowd the context**, leading to wrong tool selection.

CLASH

Contradictory information creates conflicting assumptions and unreliable output.

Context Rot

Information becomes **stale over time**. Facts true at the start of a task may be wrong by the time the agent acts.

The agent **can't distinguish** fresh information from outdated -- it treats everything as equally current.

Context rot is why context engineering isn't one-time setup -- it requires **ongoing maintenance**.

REFRESH

Periodically re-fetch and validate changing information

SUMMARIZE

Compress history into fresh summaries preserving key knowledge

PRUNE

Actively remove irrelevant or outdated context

VALIDATE

Check consistency and usefulness of retrieved info

Context Hygiene = avoid overload + detect irrelevance + prune/compress + keep memory clean

SECTION 04

Patterns & Strategies

The four fundamental operations of context engineering

The Four Context Operations

Every context engineering strategy maps to one of these.

W

WRITE

Create context that
doesn't exist yet

S

SELECT

Choose the most
relevant context from
available sources

C

COMPRESS

Reduce size while
preserving essential
information

I

ISOLATE

Separate context types
to prevent interference

WRITE Creating Context

Generating information that **doesn't exist yet**. The system actively creates new context for future decisions.

PLANS & INSTRUCTIONS

Agent writes multi-step plans, storing them as files for reference throughout execution.

SELF-UPDATING RULES

Agents write their own instruction files from user feedback -- continuous improvement without prompt engineering.

LEARNED KNOWLEDGE

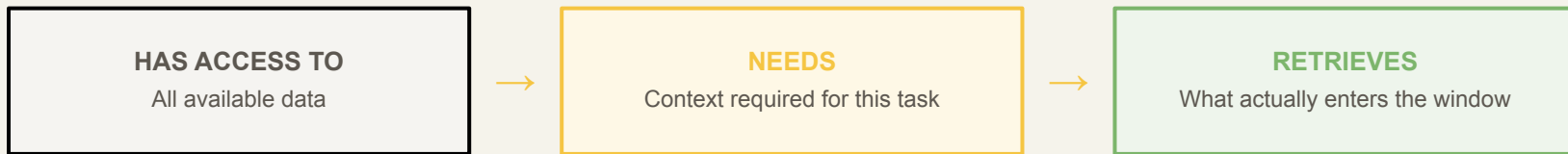
Subagents write learnings to shared files. Discoveries from one interaction become context for the next.

SCRATCH PADS

Store large intermediate results to disk rather than keeping them in the conversation history.

SELECT Choosing the Right Context

The **alignment challenge**: fitting retrieved context as closely as possible to what the model actually needs.



SEMANTIC SEARCH

Vector similarity for conceptual matches

STRUCTURAL SEARCH

Filesystem traversal (grep/glob)

ADAPTIVE RETRIEVAL

Switch strategy when initial attempts fail

DYNAMIC TOOLS

Load only tools relevant to the task

COMPRESS Reducing Without Losing

Reducing context size while **preserving essential information**. Fewer tokens, same meaning.

SUMMARIZATION

Compress accumulated history into concise summaries preserving key knowledge and decisions.

PRUNING

Actively remove irrelevant or outdated context. Use dedicated pruning models or LLM tools.

OFFLOADING

Store details externally and retrieve only when needed, instead of keeping everything active.

SUBAGENTS

Spawn subagents for token-heavy operations; return only concise summaries to the main agent.

Cognition's insight: Use a **compression layer** -- a specialized LLM that condenses action history into key decisions.

ISOLATE Preventing Interference

Separating different context types to **prevent interference**. Different concerns need different spaces.

SUBAGENT ISOLATION

Each subagent gets its own context window. Only summaries return to the main agent.

DYNAMIC TOOL LOADING

Filter and load only tools relevant to the current task. Reduces confusion.

WORKING MEMORY

Task-specific state goes to a scratchpad, keeping the main context clean.

OWN YOUR CONTEXT

Design how history, state, and data flow into the model. LLMs are stateless.

SMALL FOCUSED AGENTS

Keep agents scoped to specific tasks. Long context = higher error rates.

PRE-FETCH CONTEXT

If you know the model will need info, fetch it ahead of time.

Context Prioritization

When budget is limited, use this hierarchy.

CRITICAL

Always include first.

- Current code/content being modified
- Specific error messages
- Expected output format
- Active task requirements

HELPFUL

Include if space permits.

- Project structure
- Dependencies in use
- Coding conventions
- Related documentation

OPTIONAL

Include only if needed.

- Historical decisions
- Future plans
- Tangential examples
- Background reference

Rule: Models can ignore irrelevant info, but cannot perform without relevant info. When in doubt, prioritize recall.

The Contrarian Twist

Don't build multi-agents.

Cognition (makers of Devin) argues multi-agent frameworks create **fragile systems** through dispersed decision-making.

Parallel agents make unstated assumptions that conflict, producing incoherent outputs.

The real frontier isn't dispersing work -- it's improving how single agents manage context.

SHARE FULL TRACES

Share complete agent traces, not just individual messages.

SINGLE-THREAD + COMPRESS

One agent with a compression layer beats dispersed multi-agent systems.

CONTEXT OVER COORDINATION

The bottleneck is context quality, not parallel execution speed.

Key Takeaways

1

Context engineering is entropy reduction

High-entropy intentions into low-entropy representations machines understand

2

Systems, not prompts

The full architecture of information flow -- storage, management, and usage

3

Four operations: Write, Select, Compress, Isolate

Every strategy maps to one of these fundamental operations

4

Quality > quantity

Bigger windows don't help -- focused, relevant, well-managed context does

5

Context hygiene is continuous

Monitor, prune, refresh, and validate throughout the lifecycle



References & Sources

- 1** **LangChain** -- "How Agents Can Use Filesystems for Context Engineering"
blog.langchain.com/how-agents-can-use-filesystems-for-context-engineering
- 2** **Weaviate** -- "Context Engineering: LLM Memory and Retrieval for AI Agents"
weaviate.io/blog/context-engineering
- 3** **Weaviate** -- "The Context Engineering Guide" (eBook)
weaviate.io/ebooks/the-context-engineering-guide
- 4** **arXiv:2510.26493** -- "Context Engineering 2.0"
arxiv.org/pdf/2510.26493
- 5** **Cognition** -- "Don't Build Multi-Agents"
cognition.ai/blog/dont-build-multi-agents
- 6** **Lance Martin** -- "Context Engineering Deep Dive" (LangChain)
chatpdf.com/yt/U5GPQimKybmky52Gm3rse



UP NEXT: SESSION 2

Engineering Context in Agentic Systems

CLI Chat App

Building a chat app with prompt toolkit
and the Claude Agent SDK

Context in Conversation

Managing context across multi-turn
interactions

Agentic Retrieval

Building efficient retrieval workflows
for agent-driven systems

SESSION 2

Engineering Context in Agentic Systems

From naive agent loops to production-grade
context management strategies.

Hands-On Context Engineering

O'Reilly Live Training · Lucas Soares

Where We Left Off

Session 1 established the foundations -- now we apply them to agents.

CONTEXT = RAM

Everything the model sees at inference time. System prompts, messages, tool results, history.

FOUR OPERATIONS

Write, Select, Compress, Isolate -- every strategy maps to one of these.

CONTEXT ROT

Performance degrades well before hitting limits. Quality beats quantity.

ENTROPY REDUCTION

Find the smallest set of high-signal tokens that maximize the desired outcome.

Today: These principles become critical when context flows in not just from humans, but from **tool calls during the agent's trajectory**.

SECTION 01

From Chat to Agents

Why context engineering becomes the #1 job

The Shift: Chat Models to Agents

CHAT MODEL

Human message is the primary input
Context is **manually crafted** by the user
Prompt engineering suffices
Single turn or short conversation



AGENT

Context flows from **tool calls** during the trajectory
Context is **machine-generated** and accumulates
Context engineering is essential
Dozens to hundreds of tool calls

"With agents the game is a bit trickier -- the agent's getting context not just from the human but now context is flowing in from tool calls during the agent trajectory." -- Lance Martin

The Naive Agent Problem

500K

tokens per run (naive deep research)

\$1-2

cost per single agent run

~50-100s

tool calls in production agents

CONTEXT WINDOW LIMIT

Token-heavy tool call results rapidly fill the window. The agent literally runs out of space.

PERFORMANCE DEGRADATION

Performance drops **well before** hitting the limit. Context rot begins the moment noise exceeds signal.

COMPOUNDING ERRORS

The agent generates context that feeds back into itself. Hallucinations from early steps poison later decisions.

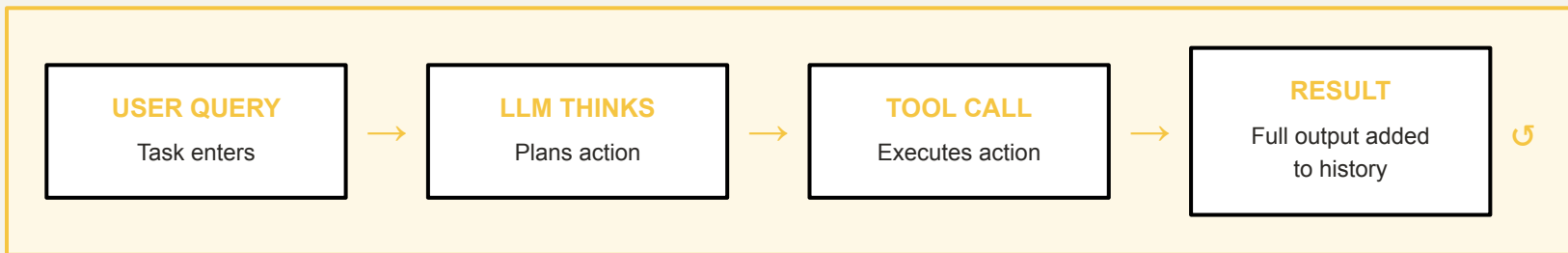
"When you put together a naive agent, context is flowing in from all these tool calls -- dozens to hundreds. And there's degradation in performance with respect to context length." -- Lance Martin

SECTION 02

The Agent Loop

How context accumulates and degrades

The Naive Agent Loop



Tool call results are **naively plumbed back** into message history. The context window grows rapidly as tool feedback accumulates.

Unlike chat, the agent **can't just start over** -- it needs continuity across dozens of steps.

Both token-heavy individual tool calls (e.g., web search results) and sheer volume of iterations contribute to the problem.

CONFLICTING INFO

Different tool results may contradict each other

POISONED HISTORY

Hallucinations from early steps corrupt later reasoning

ATTENTION DILUTION

Critical info buried under pages of tool output

The Core Engineering Challenge

"Find the smallest set of high-signal tokens that maximize the likelihood of your desired outcome."

-- Anthropic, Effective Context Engineering for AI Agents

An agent running in a loop generates data that could be relevant for the next turn. This information must be **cyclically refined**.

LEAN

Remove everything the agent doesn't need for the next decision

COMPLETE

Preserve all information critical for the current task

FRESH

Prioritize recent, validated context over stale history

STRUCTURED

Organize context so the model can navigate it efficiently

SECTION 03

Writing Context

External memory for agents

WRITE Scratchpads

Persist information **while an agent is performing a task**. Notes and plans survive context window resets.

TODO.MD PATTERN

Agent maintains a todo list file, recites goals to keep them in active attention. Survives compaction.

PLAN FILES

Anthropic's multi-agent researcher saves a plan to memory -- retrievable even if the context window limit is exceeded.

PROGRESS LOGS

Feature lists in JSON format. The model is less likely to inappropriately change or overwrite JSON files.

"Scratchpads persist information while an agent is performing a task." -- Lance Martin

WRITE Memories, State & Filesystem

CROSS-SESSION MEMORIES

Persist across sessions. Cursor and Windsurf auto-generate memories from user-agent interactions. Updated dynamically.

RUNTIME STATE OBJECTS

Pydantic models with fields as "buckets" for context. Selectively pass specific context to the LLM at certain stages.

FILESYSTEM AS MEMORY

Single interface for storing, retrieving, and updating an infinite amount of context. Agents read/write files as external memory.

INTRA-SESSION

Scratchpads live only during the task. Useful for plans, intermediate results, and todo lists.

CROSS-SESSION

Memories persist between sessions. User preferences, learned patterns, project knowledge.

STRUCTURED STATE

Schema-driven context buckets. Different agent stages access different fields, keeping context focused.

SECTION 04

Selecting Context

Agentic retrieval vs classical RAG

Three Retrieval Approaches

Benchmarked on LangGraph documentation (Lance Martin)

VECTOR STORE

Classic chunking + embeddings + similarity search

Traditional RAG pipeline. Requires indexing, chunking strategy decisions, embedding model selection.

AGENTIC SEARCH

llm.txt file with URLs + descriptions. Agent fetches docs on demand with a simple tool.

Extremely effective with good descriptions.

CONTEXT STUFFING

Feed all 3M tokens of docs directly to the agent

Brute force. Works with large windows but expensive and degrades quality.

Result: Agentic search with llm.txt and good descriptions was "extremely effective" -- a markdown file with URLs and descriptions, plus a simple fetch tool.

Agentic Retrieval: The Claude Code Way

No indexing whatsoever. Claude Code uses agentic search -- just grep/glob to explore files. No embeddings, no vector store.

Progressive disclosure: agents incrementally discover relevant context through exploration. Lightweight identifiers (file paths, stored queries, web links) are loaded dynamically at runtime.

Windsurf takes the opposite approach: code chunking, embeddings, vector search, knowledge graphs, reranking. More complex but handles very large codebases.

GREP / GLOB

Simple file search tools. No index maintenance.

LLM.TXT

Markdown file with URLs + descriptions. Agent fetches on demand.

PROGRESSIVE DISCLOSURE

Start small, expand as needed. Don't pre-load everything.

LIGHTWEIGHT IDS

File paths and links -- load full content only when needed.

"Claude Code doesn't do any indexing. It's just doing agentic retrieval -- using simple tool calls, no indexing whatsoever, and it obviously works extremely well." -- Lance Martin

Give Agents a Computer, Not 100 Tools

"Instead of binding every potential action as a separate tool to an LLM, provide agents with a shell and file system access. This significantly reduces tool bloat and context window usage."

-- Lance Martin & Chroma Conversation

THIN TOOL LAYER

Small number of atomic tools (shell, read, write, search). Agents compose these into complex actions.

TOOL SELECTION VIA RAG

Agents struggle with 100+ tools. Use RAG over tool descriptions to fetch only relevant tools per task.

SOPS AS FILES

Standard Operating Procedures stored as files, retrieved via semantic search over descriptions. Progressive disclosure.

SECTION 05

Compressing Context

Offloading and shrinking without losing signal

COMPRESS Context Offloading

Don't naively send back full tool call results. **Write to disk, return a summary or pointer.**

1

Execute tool call -- web search, file read, API call returns large result

2

Write full result to disk or agent runtime state (e.g., LangGraph state)

3

Return summary + pointer to the agent -- a URL, file path, or key facts

4

Agent **fetches full context on demand** only when needed for a decision

"Don't just naively send back the full context of each of your tool calls. You can offload it -- write those to disk and write back a summary. It could be a URL -- something so the agent knows it's retrieved a thing."

-- Lance Martin

COMPRESS

The Compression Toolkit

PROMPTED SUMMARIZATION

Compress with **high recall**. Exhaustive bullet points so the agent knows whether to retrieve full context later.

Cognition: fine-tuned model for agent-agent boundary summarization

AUTO-COMPACTION

Automatic compression when nearing limits. Claude Code auto-compacts at **95% capacity**.

Preserves decisions, bugs, implementation details; discards redundant outputs

TRIMMING

Selective removal using heuristics or learned approaches. Keep only recent N messages or LLM-scored relevance.

Lightest touch: clear verbose tool outputs after processing

PROMPT CACHING

Avoid re-sending invariant context. System prompts, tool definitions, static instructions -- **cache, don't reprocess**.

Crucial for speed and cost efficiency across turns

Risk: Summarization loses information. Always **offload to disk before summarizing** to retain raw context as a safety net.

SECTION 06

Isolating Context

Sub-agents, sandboxes, and state objects

ISOLATE Sub-Agents & Sandboxes

CLEAN CONTEXT WINDOWS

Each sub-agent explores extensively (tens of thousands of tokens) but returns only a condensed summary (1,000-2,000 tokens).

SANDBOX PERSISTENCE

Sandbox persists state over multiple turns. Can house token-heavy artifacts (images, audio, data files) -- never exposed to the LLM's window.

STATE OBJECTS

Typed fields (Pydantic models) -- different "buckets" for different context types. Agent trajectory selectively accesses relevant buckets.

EXPAND TOTAL CAPACITY

Expands the total tokens the overall system can process without any single window bloating.

SEPARATION OF CONCERNS

Detailed search context remains isolated within sub-agents. The lead agent focuses on synthesizing and analyzing.

SELECTIVE EXPOSURE

Only pass back what the LLM needs for decisions. Images, raw data, and verbose outputs stay in the sandbox.

ISOLATE Inter-Agent Communication

When isolated agents need to share context, the **boundary protocol** matters enormously.

SHARED PLAN FILES

For serial long-running tasks, agents pass context via shared plan files and progress logs on the filesystem.

BOUNDARY COMPRESSION

At agent-agent boundaries, careful compression is critical. Cognition uses fine-tuned models specifically for this.

FULL TRACE SHARING

Share complete agent traces, not just individual messages. Individual messages lose critical decision context.

Key insight: Actions carry implicit decisions. When agents only see summaries, they miss the reasoning that shaped those actions -- leading to contradictions.

SECTION 07

The Multi-Agent Debate

When parallelism helps and when it hurts

The Key Distinction: Read vs Write

READ TASKS ✓

Sub-agents collect information in parallel, then a **single write step** at the end synthesizes results.

Example: Anthropic's deep researcher -- parallelized sub-agents for research, single write at the end.

Claude Code: sequential sub-agents that **answer questions but don't write code**.

WRITE TASKS ✗

Sub-agents make **implicit decisions** that conflict when compiled together.

Example: Flappy Bird -- Subagent 1 built Super Mario background, Subagent 2 created mismatched bird assets.

Parallel agents make **contradictory choices** that are hard to reconcile.

KEEP ATOMIC

Sub-agents should be focused, parallelizable, read-only tasks

INVEST IN SUMMARIZATION

Heavily compress at agent-agent boundaries (fine-tune for it)

PREFER LINEAR

For coding: single-threaded execution; sub-agents only for info gathering

SECTION 08

Static Context Files

Do CLAUDE.md and .cursorrules actually work?

Do Context Files Actually Work?

-2%

LLM-generated files
reduce success rates

+4%

Developer-written files
marginal improvement

+20%

Inference cost increase
across benchmarks

The paradox: Context files ARE followed -- agents use mentioned tools 1.6-2.5x more. But following them requires **14-22% more reasoning tokens**.

Agents work **harder** without solving tasks **better**.

Tested across 4 agents (Claude Code, Codex, Qwen Code) and 2 benchmarks (300+ tasks).

REDUNDANCY

LLM-generated files duplicate info in READMEs and docstrings.

WHEN THEY HELP

When existing docs are removed, LLM files improve by 2.7%.

BEST PRACTICE

Human-written, focused files with minimal requirements only.

Recommendation: Include only **minimal, unusual requirements**. Don't write comprehensive overviews -- agents explore effectively on their own.

Key Takeaways

1

Agents make context engineering critical

Context flows from tool calls, not just humans -- managing it is the #1 engineering job

2

Write context externally

Scratchpads, memories, state objects, and filesystem -- persist information outside the window

3

Agentic retrieval beats naive RAG

Simple tools (grep/glob) with progressive disclosure outperform complex pipelines in many cases

4

Offload before you summarize

Always save raw context to disk, then compress -- summarization is lossy

5

Read tasks parallelize, write tasks don't

Sub-agents for research, single-threaded execution for code generation



References & Sources

- 1** **Lance Martin** -- "Context Engineering for Agents in Practice"
youtu.be/4GiqzUHD5AA
 - 2** **Lance Martin** -- "Context Engineering Discussion"
youtube.com/watch?v=_IITcWciEC4
 - 3** **Lance Martin & Chroma** -- "Conversation about Context Engineering"
youtu.be/MJScoDgIcXg
 - 4** **Cognition.ai** -- "Don't Build Multi-Agents"
cognition.ai/blog/dont-build-multi-agents
 - 5** **Anthropic** -- "Effective Context Engineering for AI Agents"
anthropic.com/engineering/effective-context-engineering-for-ai-agents
 - 6** **arXiv** -- "Context Files for Coding Agents"
arxiv.org/html/2602.11988v1
 - 7** **LangChain** -- "Filesystems for Context Engineering"
blog.langchain.com/how-agents-can-use-filesystems-for-context-engineering
 - 8** **Anthropic** -- "Effective Harnesses for Long-Running Agents"
anthropic.com/engineering/effective-harnesses-for-long-running-agents
- 

UP NEXT: SESSION 3

Diagnosing and Fixing Context Failures

Failure Patterns

Identifying context poisoning, confusion, and attention dilution in practice

Diagnostic Tools

Techniques for measuring context quality and finding degradation points

Hands-On Fixes

Applying compression, pruning, and isolation to fix real context failures

SESSION 3

Diagnosing and Fixing Context Failures

Identifying failure modes in long-context agents
and applying targeted fixes.

Hands-On Context Engineering

O'Reilly Live Training · Lucas Soares

Where We Left Off

Session 2 gave us the four levers -- now we learn when each one applies.

WRITE

Scratchpads, memories, filesystem -- store context outside the window.

SELECT

Agentic retrieval, RAG over tools -- pull only what's relevant per step.

COMPRESS

Offloading, summarization, auto-compaction -- shrink without losing signal.

ISOLATE

Sub-agents, sandboxes, state objects -- clean context boundaries.

Today: We learn to **diagnose which failure is happening** and choose the right lever to fix it.

SECTION 01

When Long Context Hurts

Why failures show up in agents

The Cost of Unchecked Context Growth

OVERFLOW

Agent hits context window limit and can no longer function

COST EXPLOSION

Naive deep research agents hit 500K tokens, \$1-2 per run

CONTEXT ROT

Performance degrades well **before** hitting limits

SELF-REINFORCING LOOP

Each turn adds context the agent itself generated. Tool feedback accumulates across turns -- a compounding feedback loop unlike chat.

MORE OPPORTUNITIES FOR ERROR

As context grows, there are more opportunities for the LLM to get confused due to conflicting information or hallucinations.

"Context windows of all sizes will be subject to context pollution and information relevance concerns." -- Anthropic

Bigger Windows Don't Solve the Problem

1M+ tokens

Gemini 2.5 Pro context window -- yet performance decreased beyond 100K tokens

The agent showed a **tendency toward repeating actions** from its vast history rather than synthesizing new strategies.

More tokens = exponentially more pairwise relationships = diluted attention.
Quality degrades gradually as token count climbs.

GEMINI 2.5 PRO

Decreased performance beyond 100K tokens despite 1M+ window. Defaulted to action repetition.

LLAMA 3.1 405B

Hit distraction ceiling at ~32K tokens. Smaller models degrade even earlier.

ATTENTION DILUTION

A model's ability to capture pairwise relationships gets stretched thin as context grows.

"As the context grew significantly beyond 100k tokens, the agent showed a tendency toward favoring repeating actions from its vast history." -- Drew Breunig

SECTION 02

The Four Failure Modes

A taxonomy for diagnosing context problems

Four Ways Long Context Fails

Drew Breunig's taxonomy -- each mode has distinct symptoms and fixes.

POISONING	DISTRACTION	CONFUSION	CLASH
A hallucination enters the context and compounds. The agent treats the wrong fact as "given" in all subsequent steps. Self-reinforcing errors	Context overwhelms the model's training knowledge. Agent favors repeating past actions rather than new strategies. Attention dilution	Irrelevant content influences the response. Models call irrelevant tools even when none are appropriate. Signal-to-noise ratio	Parts of the context disagree with each other. Agent flip-flops between contradictory approaches across turns. Contradictory sources

Source: Drew Breunig, "How Long Contexts Fail" (2025) -- also referenced in LangChain, "Context Engineering for Agents"

FAILURE 01 Context Poisoning

"When a hallucination makes it into the context, where it is repeatedly referenced." Errors compound -- the agent treats its own prior output as ground truth.

GEMINI PLAYS POKEMON

The agent hallucinated game state details that poisoned its goals section. Multiple context sections became corrupted with misinformation. The agent became **fixated on achieving impossible goals**.

WHY IT'S DANGEROUS

Mistakes in agent history "poison" context and steer the agent off track. The self-reinforcing feedback loop means errors are never corrected -- they're amplified.

SYMPTOM

An incorrect claim reappears in later steps

SYMPTOM

Later steps treat the error as "given"

SYMPTOM

Agent stuck in loops pursuing impossible goals

DIAGNOSTIC

Identify where the bad fact entered (tool output? hallucination?) and trace reuse

"An agent running in a loop generates more and more data that could be relevant for the next turn of inference." -- Anthropic

FAILURE 02 Context Distraction

"When the context overwhelms the training." As context accumulates, the model over-focuses on history rather than leveraging its training knowledge.

RESEARCH EVIDENCE

Gemini 2.5 Pro: decreased performance beyond **100K tokens**, defaulting to action repetition. Llama 3.1 405B: distraction ceiling at **~32K tokens**. Smaller models degrade even earlier.

MECHANISM

More tokens = exponentially more pairwise relationships = **diluted attention**. A model's ability to capture these relationships gets stretched thin.

SYMPTOM

Model over-weights history; misses immediate instruction

SYMPTOM

Agent repeats prior actions rather than novel decisions

SYMPTOM

Quality degrades gradually as token count climbs

DIAGNOSTIC

Correlate quality drop with token growth across turns

"A model's ability to capture these pairwise relationships gets stretched thin." -- Anthropic

FAILURE 03 Context Confusion

"When superfluous context influences the response." If you put something in the context, **the model has to pay attention to it** -- regardless of relevance.

BERKELEY BENCHMARK

Every model performs worse with more tools. Llama 3.1 8B **failed** on GeoEngine with 46 tools (within context window), succeeded with only 19 tools.

BEYOND TOOLS

Irrelevant retrieved documents, prior conversation turns, and stale tool outputs all contribute. Adding **semantically similar but irrelevant distractors** causes further degradation.

SYMPTOM

Response "pulled" by irrelevant details in context

SYMPTOM

Agent calls tools that aren't relevant to the task

SYMPTOM

Performance inversely correlated with available tools

DIAGNOSTIC

Remove unrelated context (ablation) and check if behavior improves

"If you put something in the context the model has to pay attention to it." -- Drew Breunig

FAILURE 04 Context Clash

"When parts of the context disagree." Models make early assumptions that persist and override later corrections. Once lost, they **do not recover**.

MULTI-TURN STUDY

Microsoft/Salesforce: **39% average performance drop** when identical information was presented across multiple turns vs at once. o3 dropped from 98.1 to 64.1.

IN AGENT SYSTEMS

System instructions vs retrieved knowledge vs tool feedback may contradict. Earlier tool results conflict with later ones. The model can't resolve competing signals.

SYMPTOM

Inconsistent decisions across turns

SYMPTOM

Agent flip-flops between contradictory approaches

SYMPTOM

Performance drops in multi-turn vs single-turn

DIAGNOSTIC

Locate contradictions across context sources

"When LLMs take a wrong turn in a conversation, they get lost and do not recover." -- Drew Breunig

SECTION 03

The Diagnosis Workflow

A minimal debugging loop for context failures

The Five-Step Debugging Loop

1

Inspect Trace

Walk through messages, tool calls, and tool outputs turn by turn. Look at what the agent saw, decided, and received.

2

Tag Failure

Which mode? Poisoning (bad fact propagating), Distraction (repeating past), Confusion (irrelevant influence), Clash (contradictions).

3

Pick a Lever

Use the Write / Select / Compress / Isolate framework. Each failure mode has primary and supporting levers.

4

Re-run & Evaluate

Test whether the fix improved performance. Track token usage per step. Don't just optimize for fewer tokens -- measure outcome quality.

5

Iterate

Context engineering is iterative, not one-shot. The curation phase happens each time we decide what to pass to the model.

"An agent can waste context by misusing tools, chasing dead-ends, or failing to identify key information." -- Anthropic

SECTION 04

Mapping Failures to Fixes

The four levers applied to each failure mode

Recap: The Four Levers

From Session 2 -- now we map each lever to specific failure modes.

WRITE	SELECT	COMPRESS	ISOLATE
Save context outside the window	Pull only what's relevant at each step	Summarize or trim messages and tool outputs	Keep token-heavy info out of the main stream
Scratchpads, memories, filesystem, state objects	Agentic search, RAG over tools, dynamic loading	Auto-compaction, offloading, prompted summarization	Sub-agents, sandboxes, state schemas

Key question: Which failure mode am I seeing? That determines which lever(s) to pull first.

Fixing Poisoning & Distraction

POISONING FIXES

Primary: Isolate + Select

Isolate tool outputs in separate state fields until validated. Don't let raw tool outputs flow directly into history.

Select only validated facts into the next prompt -- control exposure per step.

Supporting: Write a scratchpad that stores claims + sources. Only select verified ones forward.

DISTRACTION FIXES

Primary: Compress + Select

Compress message history periodically. Claude Code auto-compacts at 95% capacity -- summarize contents, reinitiate with summary.

Select a smaller working set each step. Don't dump full history.

Supporting: Write a persistent plan/scratchpad so you can compress history without losing intent.

"Don't just naively send back the full context of each of your tool calls. You can offload it -- write those to disk and write back a summary." -- Lance Martin

Fixing Confusion & Clash

CONFUSION FIXES

Primary: Select + Isolate

Dynamic tool loading: LLM-powered recommender improved Llama 3.1 8B performance by **44%** by selecting relevant tools per step.

Isolate "nice-to-have" context in separate fields -- inject only when required.

Supporting: Drew Breunig's "Tool Loadout" -- DeepSeek-v3 fails at 100+ tools, select subset based on task.

CLASH FIXES

Primary: Isolate + Select

Context quarantine: isolate tasks into separate threads with dedicated contexts. Anthropic's multi-agent system outperformed single-agent by **90.2%**.

Select with precedence rules -- decide what gets included for this step.

Supporting: Compress conflicting sections into a single resolved summary after deciding precedence.

Key insight: RAG over tool descriptions (embedding-based semantic similarity) to fetch relevant tools for a given task is one of the highest-impact fixes for confusion.

SECTION 05

Observability & Evaluation

Measuring what matters in context engineering

Measuring What Matters

TRACK TOKEN USAGE

Monitor where context grows (per-step token counts). Identify which tool calls contribute the most tokens. Even **approximate measurement** (string length) provides useful signal.

VALIDATE FIXES

Use evaluation harnesses to test context engineering changes. Don't just optimize for fewer tokens -- **measure outcome quality**. A fix that reduces tokens but degrades results is worse.

ITERATE CONTINUOUSLY

Context engineering is not a one-time optimization. Create **dedicated stages** for compression/pruning to collect optimization data.

PROVENCE (PRUNING)

Context pruning tool that reduced a Wikipedia article by 95% while preserving relevant details for QA

CONTEXT OFFLOADING

Pairing offloading with domain-specific prompts yielded up to 54% performance improvements

PROMPTED SUMMARIZATION

Engineer summaries for high recall -- agent needs to know whether to fetch full context

"I spent a lot of time prompt engineering to make sure my summaries capture with high recall what's in the document but compress the content significantly." -- Lance Martin

Quick-Reference: Failures to Fixes

The debugging checklist in one view.

FAILURE MODE	PRIMARY FIX	SUPPORTING FIX	KEY SIGNAL
Poisoning	Isolate + Select	Write (claims + sources)	Bad fact propagates across steps
Distraction	Compress + Select	Write (persistent plan)	Agent repeats past actions
Confusion	Select + Isolate	Tool Loadout (dynamic)	Irrelevant tools/docs selected
Clash	Isolate + Select	Compress (resolved summary)	Flip-flopping between approaches

Core principle: "Find the smallest set of high-signal tokens that maximize the likelihood of your desired outcome." -- Anthropic

Key Takeaways

1

Four distinct failure modes

Poisoning, Distraction, Confusion, and Clash -- each has different symptoms and requires different fixes

2

Bigger context windows don't solve the problem

Performance degrades well before hitting limits -- Gemini 2.5 Pro showed decreased quality beyond 100K tokens

3

Diagnose before you fix

Trace the agent trajectory, tag the failure mode, then pick the right lever -- don't apply fixes blindly

4

Each failure maps to specific levers

Poisoning needs Isolate+Select, Distraction needs Compress+Select, Confusion needs Select+Isolate, Clash needs Isolate+Select

5

Context engineering is iterative

Measure token usage and outcome quality, iterate until performance stabilizes -- not a one-shot optimization



References & Sources

- 1** **Drew Breunig** -- "How Long Contexts Fail"
dbreunig.com/2025/06/22/how-contexts-fail-and-how-to-fix-them.html
- 2** **Drew Breunig** -- "How to Fix Your Context"
dbreunig.com/2025/06/26/how-to-fix-your-context.html
- 3** **LangChain** -- "Context Engineering for Agents"
blog.langchain.com/context-engineering-for-agents
- 4** **Anthropic** -- "Effective Context Engineering for AI Agents"
anthropic.com/engineering/effective-context-engineering-for-ai-agents
- 5** **Lance Martin** -- "Context Engineering for Agents in Practice"
youtu.be/4GiqzUHD5AA
- 6** **Lance Martin** -- "Context Engineering Discussion"
youtube.com/watch?v=_lITcWciEC4



UP NEXT: SESSION 4

Context Engineering Patterns in Modern AI Apps

Production Patterns

Real-world context engineering patterns from Claude Code, Cursor, and Windsurf

Architecture Recipes

Combining write, select, compress, and isolate into cohesive systems

Code Walkthrough

Building a context-aware agent with proper failure handling from scratch

SESSION 4

Context Engineering Patterns in Modern AI Applications

How production systems manage what enters the context window
and when.

Hands-On Context Engineering

O'Reilly Live Training · Lucas Soares

Where We Left Off

Session 3 taught us to diagnose context failures -- now we see how production systems prevent them.

POISONING

Hallucinations enter context and compound. Fix: Isolate + Select verified facts forward.

DISTRACTION

Context overwhelms training knowledge. Fix: Compress history + persistent plan.

CONFUSION

Irrelevant tools/docs influence output. Fix: Dynamic tool loading.

CLASH

Context sources contradict each other. Fix: Sub-agent isolation.

Today: How do **production AI systems** (Claude Code, Cursor, Manus) implement these fixes at scale?

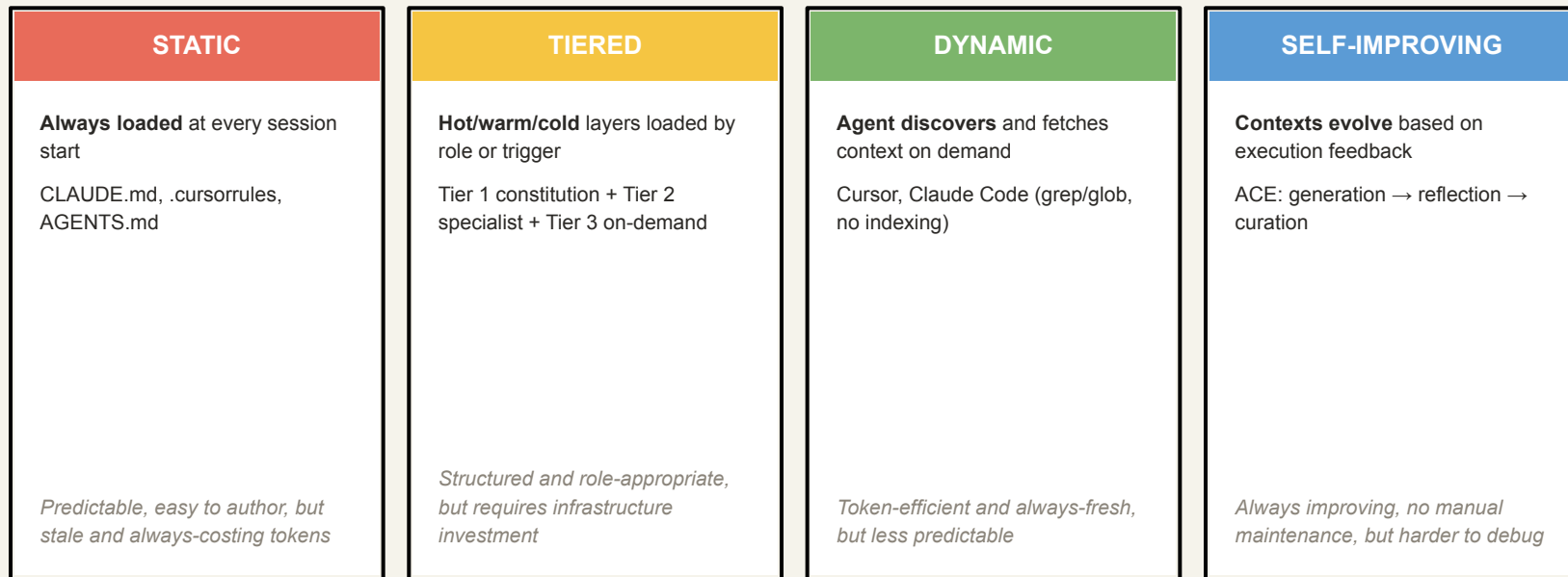
SECTION 01

From Theory to Production

How real systems manage context

The Static-to-Dynamic Spectrum

Production systems combine multiple points on this spectrum.



Less dynamic → More dynamic | More predictable → Less predictable

"Context Engineering is not about adding more context. It is about finding the minimal effective context required for the next step." -- Peak Ji, Manus

SECTION 02

Static Context Files

Do they actually work?

The ETH Zurich Study

Controlled evaluation across 300+ tasks, 4 agents, 2 benchmarks

-3%

LLM-generated context
files
(average success rate
change)

+4%

Human-written context
files
(average success rate
change)

+20%

Cost increase
(both file types)

THE REDUNDANCY FINDING

LLM-generated files duplicate information already in READMEs and code comments. When existing docs are removed, LLM files improve by **+2.7%** -- they only help when no other docs exist.

SCOPE

4 agents: Claude Code, Codex, Qwen Code. 2 benchmarks: AGENTbench + SWE-bench.

TOOL USAGE

Agents use tools mentioned in context files 1.6-2.5x more often

REASONING COST

Following instructions costs 14-22% more reasoning tokens

"LLM-generated context files have a small negative effect on agent performance (a decrease of 3% on average)." -- Gloaguen et al.

The Paradox & What Works

THE INSTRUCTION PARADOX

Agents **do follow** the instructions -- they use mentioned tools 1.6-2.5x more. But following them costs **14-22% more reasoning tokens**. Agents work **harder** without solving tasks **better**.

THE STALENESS PROBLEM

Static files rot over time. Four failure patterns: vague instructions, missing feedback commands, outdated information, file divergence from codebase evolution.

INSTRUCTION LIMITS

Frontier LLMs can follow **~150-200 instructions** consistently. Claude Code's system prompt already uses ~50. Budget your static context carefully.

KEEP IT MINIMAL

10-20 lines focused on what the agent genuinely cannot infer from the codebase

SPECIFIC TOOLING ONLY

Custom build commands, package manager preferences -- not general coding advice

DON'T DUPLICATE

If it's in your README or code comments, don't repeat it in CLAUDE.md

PROGRESSIVE DISCLOSURE

Brief descriptions in static file; detailed content in separate files the agent loads on demand

REVIEW QUARTERLY

Treat context files like dependency versions -- audit them regularly for staleness

SECTION 03

Dynamic Context Loading

Priming, discovery, and agent skills

Context Priming & Loading Modes

Asking the LLM to dynamically fetch reference materials before substantive work begins.

WHAT IS CONTEXT PRIMING?

At session start, ask the model to **find and load reference materials**. It uses its own tools (web search, file reading) to populate context with accurate, current documentation -- not stale training weights.

STATIC VS. PRIMING

Static: loaded before the model sees any task → always-on, always-costing tokens.

Priming: loaded on demand, targeted to current task → context-efficient, always-fresh.

PROGRESSIVE DISCLOSURE

Brief descriptions in CLAUDE.md; detailed content in separate files. The static file becomes a **routing table**, not a content dump.

LLM-CONTROLLED

Agent autonomously decides when and what to load.
Prerequisite for unsupervised agents.

HUMAN-CONTROLLED

Slash commands, file selections, @-mentions. User drives context loading.

DETERMINISTIC HOOKS

Triggered at lifecycle points (e.g., always load CLAUDE.md at session start).

"All this means is that, at the beginning of our conversation, I'll ask Claude to find reference materials about the topic I plan to ask about." -- Chris Dzombak

Cursor: Dynamic Context Discovery

Production A/B test results from Cursor's engineering team.

46.9%

Token reduction in runs
utilizing MCP tools

5

Implementation strategies
for dynamic loading

Files

As a simple and powerful
primitive for LLM interfaces

TOOL RESPONSE FILES

Long shell/MCP outputs written to files, not injected into context. Agent reads selectively.

CHAT AS FILES

Previous chats become accessible files the agent can re-read selectively during summarization.

AGENT SKILLS

Domain capabilities bundled as discoverable files. Agent finds via grep/search, not static injection.

MCP OPTIMIZATION

Tool descriptions synced to organized folders, dramatically reducing static prompt bloat.

TERMINAL ACCESS

Terminal outputs become filesystem-accessible. Agent searches logs dynamically.

"Providing fewer details up front, making it easier for the agent to pull relevant context on its own." -- Cursor

Agent Skills: Loadable Knowledge Modules

Structured knowledge that an agent loads on demand to govern reasoning about context.

PROGRESSIVE DISCLOSURE

Skills stored as files with brief name/description exposed statically. When a task activates a skill, the agent loads the full content. This **is** context engineering in practice: load what you need when you need it.

KEY PATTERNS

n^2 attention budget: attention depletes as context grows. Place critical info at beginning and end.

Tool outputs consume 83.9% of total context in typical agent trajectories.

Compaction triggers at 70-80% utilization -- don't wait for the limit.

SKILLS MAP TO FOUR LEVERS

Write: filesystem-context, memory-systems • **Select:** context-fundamentals, tool-design

Compress: context-compression, optimization • **Isolate:** multi-agent-patterns, hosted-agents

FOUNDATIONAL (3)

context-fundamentals, context-degradation, context-compression

ARCHITECTURAL (5)

multi-agent-patterns, memory-systems, tool-design, filesystem-context, hosted-agents

OPERATIONAL (3)

context-optimization, evaluation, advanced-evaluation

METHODOLOGY (1)

project-development

COGNITIVE (1)

bdi-mental-states

Consolidation Principle: "If a human engineer cannot definitively say which tool should be used, an agent cannot do better." Vercel reduced from

[1] Kojima, Shikun, et al. "Large Language Models as Tool-Augmented Engineers." arXiv preprint arXiv:2308.07430, 2023.

SECTION 04

Production Agent Harness

Lessons from Manus's five strategies

The Three Context Problems (Manus)

Production-observed degradation patterns from building autonomous agents at scale.

CONTEXT ROT

Performance degradation as context fills.

Effective window is "much smaller" than advertised -- around **256K usable** in practice even with 1M+ windows.

Gradual quality decline

CONTEXT POLLUTION

Too much irrelevant or redundant information.

Degrades reasoning quality by diluting the signal-to-noise ratio across the entire context window.

Noise drowns signal

CONTEXT CONFUSION

LLM can't distinguish instruction types.

System instructions, user data, and structural markers blur together when context is crowded.

Role boundaries dissolve

Key insight: These three problems map directly to Session 3's failure taxonomy -- Rot $\approx$ Distraction, Pollution $\approx$ Confusion, Confusion $\approx$ Clash.

Manus Strategies 1-3

1

Context Compaction & Summarization

Two-level approach: (a) **reversible compaction** -- store file paths, not full contents; (b) **lossy summarization** triggered at ~128K tokens. Preserve recent tool calls in raw format; summarize older turns as JSON.

2

Communication Over Shared Context

Spin up **fresh sub-agents** for discrete tasks with minimal context rather than sharing full conversation history. Only share full history when sub-agents must understand the problem trajectory. Inspired by Go's concurrency principle.

3

Hierarchical Action Spaces

Limit visible tools to ~**20 atomic tools** (file_write, browser_navigate, bash). Level 2: sandbox utilities via general-purpose tools. Level 3: code libraries for complex chains -- avoids multiple expensive LLM roundtrips.

"Share memory by communicating, don't communicate by sharing memory." -- Peak Ji, Manus

Manus Strategies 4-5

4 Agent-as-Tool Pattern

Treat sub-agents as **deterministic functions** with defined inputs, outputs, and JSON schemas -- not as anthropomorphized "team members."

MapReduce pattern flattens hierarchical complexity. Avoids over-engineering agent "org charts."

Each sub-agent is a black box: send input, get output. No shared state, no complex coordination protocols.

THE "INTERN TEST"

Binary success metrics on real environments rather than subjective evaluation. Can an intern verify it passed?

KV CACHE MATTERS

Static tool definitions at the start of context enable KV cache reuse. Dynamic injection breaks this.

5 Implementation Best Practices

Avoid dynamic RAG-based tool retrieval -- it breaks KV cache and eliminates caching savings.

Don't fine-tune prematurely -- build flexible harnesses instead. Fine-tuning locks you into a specific model version.

Define pre-rot thresholds proactively -- set limits before degradation sets in, not after.

PRE-ROT THRESHOLDS

Set compaction triggers at 70-80% utilization. Don't wait for the context limit.

SECTION 05

Tool Design as Context Engineering

Every tool definition consumes tokens at every turn

Tool Design & Token Economics

THE CONSOLIDATION PRINCIPLE

Fewer, more powerful tools beat many narrow ones. **Vercel**: reduced from 17 tools to 2 primitives → success rate jumped from **80% to 100%**. One tool handles listing, lookup, and creation.

TOOL DESCRIPTIONS AS PROMPTS

Each description must answer: what does it do, when to use it, what inputs, what it returns. **Anti-patterns**: vague descriptions, cryptic parameter names (x, val, param1), missing error recovery guidance.

HIERARCHICAL ACTION SPACE

Level 1: ~20 atomic tools visible to LLM. **Level 2**: sandbox utilities via general-purpose tools. **Level 3**: code libraries for complex chains (0 LLM calls for routine operations).

ARCHITECTURE	TOKEN MULTIPLIER
Single agent chat	1x baseline
Single agent + tools	~4x baseline
Multi-agent system	~15x baseline

83.9%

Of total context consumed by tool outputs in typical agent trajectories

MCP BEST PRACTICE

Use fully qualified names (ServerName:tool_name) to avoid "tool not found" errors

VERCEL RESULT

17 tools → 2 primitives = 80% → 100% success rate

"If a human engineer cannot definitively say which tool should be used, an agent cannot be expected to do better." -- Koylan

SECTION 06

The Full Spectrum

A synthesis of static to self-improving

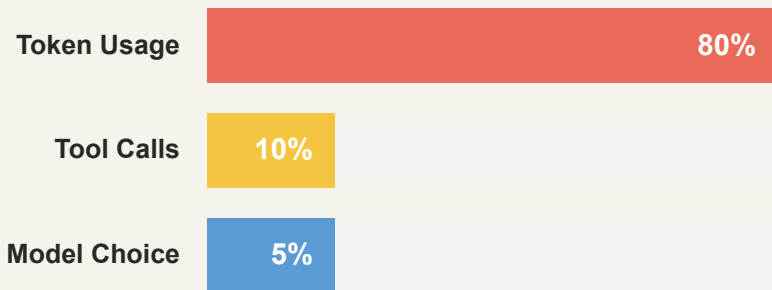
The Static-to-Dynamic Spectrum: Full View

STATIC FILES	TIERED	DYNAMIC	SELF-IMPROVING
CLAUDE.md, .cursorsrules -- always loaded at session start ETH study: -3% LLM-generated, +4% human-written Always-on, always-costing tokens, prone to staleness <i>Best for: stable conventions, build commands</i>	Hot/warm/cold layers. Tier 1 always (~660 lines), Tier 2 by role (~9,300), Tier 3 on-demand (~16,250) 108K-line C# system, 24.2% codebase as context infrastructure <i>Best for: large codebases, multi-agent</i>	Agent discovers and fetches context on demand via filesystem Cursor: 46.9% token reduction Files as a primitive for LLM interfaces <i>Best for: large codebases with many tools</i>	ACE: contexts evolve via generation → reflection → curation +10.6% on agent benchmarks, +8.6% on finance Works without labeled supervision <i>Best for: repetitive workflows with learnable patterns</i>
+ Predictable, easy to author - Stale, adds token cost	+ Structured, role-appropriate - Significant infrastructure investment	+ Token-efficient, always-fresh - Requires tool infrastructure	+ Always improving, no maintenance - Harder to debug, less transparent

Research consensus: "Minimal static + dynamic on-demand" beats either extreme. "Build context files up gradually, not all at once." -- Martin Fowler

Where Performance Actually Lives

Variance in agent performance explained by:



Optimizing **token usage** (context engineering) has **16x more impact** than upgrading your model.

WHAT THIS MEANS

The biggest lever for improving agent performance isn't picking a better model -- it's **engineering what goes into the context window**.

PRACTICAL IMPLICATION

Before upgrading from Haiku to Opus, first optimize what context your agent sees. You'll likely get better results at lower cost.

Bottom line: Context engineering is not a nice-to-have -- it's the **primary determinant** of agent performance.

Pattern Selection Guide

When to use which pattern -- a quick reference.

SITUATION	RECOMMENDED PATTERN	WHY
Stable project conventions	Minimal static file (CLAUDE.md)	Rarely changes; agent can't infer from code (linter config, build commands)
Topic-specific research tasks	Context priming at session start	Fetches current docs; avoids stale training knowledge
Large codebases, many tools	Dynamic discovery + tool consolidation	Avoids confusion from irrelevant tools and documents
Long-running multi-step agents	Tiered architecture + filesystem context	Hot context for orientation; cold context on demand
Multi-agent systems	Communication over shared context	Fresh sub-agent windows; share by communicating (Manus)
Repetitive learnable workflows	Self-improving contexts (ACE)	Contexts evolve without manual maintenance

The meta-principle: The instructions you give about how to manage context are themselves context. This recursive nature is why production systems converge on **progressive disclosure**.

Key Takeaways

1

Static files help -- but only when minimal and human-written

LLM-generated files hurt performance by -3%. Keep to 10-20 lines of what agents can't infer.

2

Dynamic loading beats static dumping

Cursor's dynamic discovery reduced tokens by 46.9%. Let agents pull context on demand.

3

Fewer, better tools outperform many narrow ones

Vercel went from 17 tools to 2 primitives: 80% to 100% success. Tool outputs consume 83.9% of context.

4

Token usage is 16x more important than model choice

80% of agent performance variance comes from token usage. Optimize context before upgrading models.

5

Production systems converge on progressive disclosure

Minimal static index + dynamic loading. The research consensus beats either extreme alone.



References & Sources

- 1 **Gloaguen et al.** -- "Evaluating AGENTS.md" (arXiv 2602.11988)
arxiv.org/html/2602.11988v1
- 2 **Chris Dzombak** -- "LLM Q&A Technique: Context Priming"
dzombak.com/blog/2025/10/llm-q-a-technique-context-priming
- 3 **Cursor** -- "Dynamic Context Discovery"
cursor.com/blog/dynamic-context-discovery
- 4 **Koylan** -- "Agent Skills for Context Engineering" (GitHub)
github.com/muratcankoylan/Agent-Skills-for-Context-Engineering
- 5 **Peak Ji / Manus via philschmid.de** -- "Context Engineering Part 2"
philschmid.de/context-engineering-part-2
- 6 **Martin Fowler** -- "Context Engineering for Coding Agents"
martinfowler.com/articles/exploring-gen-ai/context-engineering-coding-agents.html
- 7 **HumanLayer** -- "Writing a Good CLAUDE.md"
humanlayer.dev/blog/writing-a-good-claude-md
- 8 **arXiv 2602.20478** -- "Codified Context: Infrastructure for AI Agents"
arxiv.org/html/2602.20478v1
- 9 **arXiv 2510.04618** -- "Agentic Context Engineering (ACE)"
arxiv.org/abs/2510.04618
- 10 **Packmind** -- "Writing AI Coding Agent Context Files"
packmind.com/evaluate-context-ai-coding-agent



UP NEXT: SESSION 5

Tools and Techniques for Modern Development

Claude Code Deep Dive

Hooks, custom commands, memory systems, and context management in practice

Cheatsheet

A compact reference card for every pattern and technique from all 5 sessions

Putting It All Together

Building your own context engineering playbook for production agents